第一题

1. Flowchart

[10 points] Write a function with arguments , , and to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random , , and values. `Print_values a b c a b c`

一．代码如下

```
def Print_values(a,b,c):

    a = input("请输入一个数:")
    b = input("请输入第二个数:")
    c = input("请输入第三个数:")
    if a > b:
       if b > c:
          print(a,b,c)
       elif a > c:
            print(a,c,b)
       else:
            print(c,a,b)

    elif b > c:
         print("done")

    else:
         print(c,b,a)

Print_values(a,b,c)
```

二．做题思路

第一步：题目要求输入三个数

第二步：根据 if-elif-else 循环去解流程。a>b 判断为 true 之后还需要判断 b>c 或者 a>c。就用了两个 if 嵌套的 loop。

第三步：左侧的 false，我认为都是同层的，就用了一个 if-elif-else 循环完成。

三．运行结果

```
请输入一个数: 1
请输入第二个数: 5
请输入第三个数: 3
done
```

```
请输入一个数: 5
请输入第二个数: 6
请输入第三个数: 7
7 6 5
```

```
请输入一个数：1
请输入第二个数：2
请输入第三个数：3
3 2 1
```

```
请输入一个数：9
请输入第二个数：6
请输入第三个数：3
9 6 3
```

说明：代码都是自己手动输入的。只有一个代码问了 gpt。询问 gpt：“如何编写 Print_values 函数来包含 a,b,c”。之前学过 def 定义函数，我忘记了。是依靠 gpt 的提醒完成的 def Print_values 这一行代码的编写。

第二题

2. Matrix multiplication

2.1 [5 points] Make two matrices (rows and columns) and (rows and columns); both are filled with random integers from and . `M1 5 10 M2 10 5 0 50`

2.2 [10 points] Write a function to do matrix multiplication, *i.e.*, . Here you are **ONLY** allowed to use loop, operator, and operator. `Matrix_multip M1 * M2 for * +`

一．代码

```
#第二题代码
import numpy as np
M1 = np.random.randint(0,51,(5,10))
M2 = np.random.randint(0,51,(10,5))

MM = np.zeros((5, 5), dtype=int)

for i in range(5):
    for j in range(5):
        for k in range(10):
            MM[i, j] += M1[i, k] * M2[k, j]
print(MM)
```

```
[[7288 8354 6894 8291 7893]
 [4380 5317 6050 7127 4679]
 [6658 6698 6294 7480 7834]
 [5062 5194 3085 5133 4665]
 [4373 4998 5365 4994 5212]]
```

二．思路

第一步：求两个随机矩阵。10.10 号晚上，老师介绍了矩阵，在课上和课后练习里面都用到了 np.random.randint 函数求随机的整数矩阵。利用 np.random.randint(5,10,(5,10))就可以得到一个 5x10 的随机整数矩阵。

第二步：要求计算两个矩阵相乘。因为行列数不一样不能直接相乘。需要用循环提取元素，5x10 和 10x5 的两个矩阵，每一行和每一列的数就要乘 10 次，最后得到一个 5x5 的矩阵。

三．两次不同的运行结果

```
print(MM)
```

```
[[7288 8354 6894 8291 7893]
 [4380 5317 6050 7127 4679]
 [6658 6698 6294 7480 7834]
 [5062 5194 3085 5133 4665]
 [4373 4998 5365 4994 5212]]
```

```
print(MM)
```

```
[[6730 5652 5468 4712 5524]
 [8559 8656 6283 6234 6303]
 [6430 8330 7019 5693 6735]
 [7216 7040 4902 4651 6247]
 [8282 8006 6637 5209 7397]]
```

说明：第一步的代码都是自己写的，老师上课也用过随机函数。第二步里面的 for 循环，我实在想不出来就让 gpt 给我举了一个例子，询问 gpt“怎么用 for 循环求矩阵的乘法”。根据 gpt 举的例子，我改了一下代码就得到这道题的结果了。



第三题

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function with an argument to print the k `Pascal_triangle k` th line of the Pascal triangle. Report and `.Pascal_triangle(100) Pascal_triangle(200)`

一．代码

```
def Pascal_triangle(k):
    num = 1
    for j in range(k):
        print(num,end='')
        num = num * (k - 1 - j) // (j + 1)

Pascal_triangle(100)
```

二．思路

1.题目要求解帕斯卡三角，输出 100 和 200 行的结果

2.数学问题就可以利用公式，把数学公式写成对应的代码形式，即可求解这道题。

数学公式用代码表示：num = num * (k - 1 - j) // (j + 1)

3.题目要求给 Pascal_triangle 定义一个 k，就要用 def 函数。代码弄好后，最终输入 Pascal_triangle（100）和（200）就能得到结果。

三．结果

1. 100 行的值

```
Pascal_triangle(100)
```

```
1994851156849376437671523144112052925614887031544171200862756173103094564415579278510796126050526132804924370524973896618617197482530438000770702498
2962153377006474903441130522928399324306551961194453787749425144898858450330806107196674080761936594428786696323047746376161305471473908437922457190
1217043857188997619146258135816088501224606298174300842802538761818894522902528407616285176853642107196237061721399667836569723427057428359914586546
5003098147672881170194648328344718912820560637875127661376774632457640004317357624192725689724850091743849514095420719744392610510239922557369338327
3503615787010261407757711793649572175876199757263126541093257275711324401291221546186149211810306587246883515430371713505892127392912549849365832112
4600506947888824774048748168690076042183211868699725888281149874753368163901091452742930164937070322172642375071243492884049536827651812046361280818
5242668323379665916777489877819632612839674339023040098565708730672447391482600239409357992069284846741061502593601378247417250445672272782096667406
2486285044567227278209666740624862848467410615025936013782474172447391482600239409357992069283967433902304009856570873067233796659167774898778196326
1282765181204636128081852426683221726423750712434928840495368163901091452742930164937070321186869972588828114987475336882477404874816869007604218325
4984936583211246005069478883515430371713505892127392912215461861492118103065872468812654109325727571132440129127117936495721758761997572633832735036
1578701026140775719744392610510239922557369397248500917438495140954207457640004317357624192725682056063787512766137677463288117019464832834471891283
5991458654650030981476721399667836569723427057428517685364210719623706172181889452290252840761628606298174300842802538761914625813581608850122457190
1217043857188997616130547147390843792244287866963230477463761071966740807619365942514489885845033080655196119445378774941130522928399324306215337700
6474903443800077070249829661861719748253049243705249738961260505261328041557927851079617310309456441712008627561488703154411205292567152314437643761
568494851991
```

2. 200 行的值

```
Pascal_triangle(200)
```

```
1199197011293699633912512472258789799363675112203959847089528950363301361122550215450664213284540935626163664616203348485845741232051912627816825854
1490058933858410972062265364012126161353221012728228162226415562041646374623865603616752080125215036278855641693821434882853668195403616135877879673
5007338614764145222900917061506604753287612378523459120956991548018324100153507987433197477070330076770746561468507650149628192324565214145076905610
1097273410376395649901553833927076809138726322647810879629503973514030389937681696560304355200694140471323923032104217275838962328354343238469557686
1452255753319166029629978545842632348229449268808607501236545093108248189841215891705337637770890712049335299666638343145302259829717623825900985043
7359403955349931489584373269471102246252990031076120253743264881256829498116090695300364456691050396301163933906252885761192388252492589624981641347
6683823298321822592662584573267221641999107962993244246121055615447597181554211540918384100315990397178309924696057185334822765621640126396158871323
9698784228741339291062486360490170469769280761235835048470603119352239275582603386558657208395699442465545918488736341039239927839344585610421503974
5835352309743000482142896122217126187671504553416248106689255721626997553208121242420184901732263235717707350283820919987066676810235814927757681162
7253669347711916506428088408438467412655303236819636619601087735603688946626721813473401268113464594480811515266860347570217040690499665132340393783 4
4243454580058104271065112207149899539699448379868475975145659951693896112134614412380428304538885643162156841678559031970376774871215967850504181489
7072397277863173164144252564265448042122543723143513438889364448755919455717478288039655957970882874445207083244558110603832551700321044143891925127
3914305324997717208558384265615151112563609920226880170976517099536154804367543560817703448838080555465246183886691967827279658468717864032562112151
4547806774778441077414638075116001512183535444928353394488247448302918063415550860400352842824936112301823251453507428541903412255995015680171336502
6424996212272097716169578681727244076309941715555544136543568425599585256386076094703561659438415084303102641155082904399118669820411701245068526306
6320541440931123990183399058618527039319948936038623291588816838856948707342052634164888231346562991351144258680325097073196687708850751401942309986
4608634283908418579587747869187687905416164486123307620776970184523398900743503998135633500883358764756743910611279846626906168112172498196617650164
0523420262524405420948286592825740779748925211202361790370073410403612436521454784573876135294029767921375320717690193962730887760381418392424464627
4494180963718720179652951552420305130915671418956036996830241230463318749004901607514183573850726297133575765081163566896105531248341502679190305956
4178771618892929418019392781601721825630835043713104991920502206325562147997821114538402750448734970264632622259821061965948625249399116845301604301
9839187996446817937910021738441760548772652853221384065891146098070507151525153324434828519321537860699237816098836719147105760727287729986652242778
9823067910488567240145204562697599821315398023066810085070356722322652024076020895290729654608433191422831565353705246455163503583952402945480741409
1435984137306883049956427877533182288184607604067568642898341159714199521944993982897373629935035017240550318110509775568667215229439616832980232902
8735635611560729491448815283893349564373908329290229611057297514488044094750038754169057412069855461656562983846033878872571435601205909584143969821
6081617967847198925216767948220145455440150399959370764775503939280273150458505512499129487207365601838221725642680589492591203338505603956263878249
2880900240220182602302255147532629056224062759155200838163925716277602584752287896038688426515007847693259648009839315649712824029738547828481305339
```

说明：代码一半是自己写的，数学公式部分是借助 gpt 完成的，首先询问 gpt：杨辉三角有没有数学公式？然后根据公式我自己编写了代码。Gpt 教我用 end=””使输出的结果都在一行，还教我用//做除法，会得到取整的结果，

二、递推公式（适合写 Python）

还有一个公式，可以利用当前数字计算下一个数字：

$$a_{r+1} = a_r \frac{n-r}{r+1}$$

第四题

4. Add or double

[20 points] If you start with `1` RMB and, with each move, you can either double your money or add another `1` RMB, what is the smallest number of moves you have to make to get to exactly x RMB? Here x is an integer randomly selected from `1` to `100`. Write a function `Least_moves` to print your results. For example, `Least_moves(2)` should print `1`, and `Least_moves(5)` should print `3`.

一.代码

```python
# 第四题代码
import random
x = random.randint(1, 100)
print(x)
def Least_moves(x):
    time = 0

    while x > 1:
        if x % 2 == 0:
            x = x / 2
        else:
            x = x - 1

        time = time + 1

    print(time)

Least_moves(x)
```

```
100
8
```

二.思路

第一步：题目要求生成随机的 x 元，于是用 random.randint 函数得到随机数 x。

第二步：初始为 1 元，通过翻倍或者加一元达到 x 元。逆向思维，第一步已经生成随机的 x，只有两种情况，偶数或者奇数。因为要不断做除法以及减法，于是利用 while 循环。直至最后算出次数。

第三步：随机数 x 为偶数时，就除以 2，循环次数 time 累计加一。x 为奇数时，就加一，使 x 变成偶数，继续做除法。循环次数 time 累计加一。最后得到总的循环次数就是达到 x 元需要的次数。

三．运行结果

```
# 第四题代码
import random
x = random.randint(1, 100)
print(x)
def Least_moves(x):
    time = 0

    while x > 1:
        if x % 2 == 0:
            x = x / 2
        else:
            x = x - 1

        time = time + 1

    print(time)

Least_moves(x)
```

```
21
6
```

```
# 第四题代码
import random
x = random.randint(1, 100
print(x)
def Least_moves(x):
    time = 0

    while x > 1:
        if x % 2 == 0:
            x = x / 2
        else:
            x = x - 1

        time = time + 1

    print(time)

Least_moves(x)
```

```
20
5
```

说明：代码是自己写的。思考了很久想不出来正向怎么写代码，于是用逆向的方法写了 while 循环。循环次数 t=t-1 这一步是 gpt 教我的。

第五题

Insert `+` or `-` operation anywhere between the digits `123456789` in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function `Find_expression`, which should be able to print every possible solution that makes the expression evaluate to a random integer from `1` to `100`. For example, `Find_expression(50)` should print lines include:

$$1-2+34+5+6+7+8-9=50$$

and

$$1+2+34-56+78-9=50$$

5.2 [5 points] Count the total number of suitable solutions for any integer i from `1` to `100`, assign the count to a list called `Total_solutions`. Plot the list `Total_solutions`, so which number(s) yields the maximum and minimum of `Total_solutions`?

一．5.1 的解题思路：

本题利用双层 `for` 循环找出所有符合条件的表达式。首先，数字 1～9 之间共有 8 个空隙，每个空隙有三种选择：添加加号、添加减号或不添加符号，因此共有 (3^8=6561) 种组合。外层 `for` 循环用于遍历全部 6561 种组合，内层 `for` 循环依次处理数字 2～9。通过取余运算和整除运算，确定每个空隙对应的操作。当选择不添加符号时，将相邻数字合并；当选择加号或减号时，先计算前面已经组成的数字，再更新运算符和当前数字。同时，利用字符串拼接保存完整的数学表达式。每次生成表达式后，计算其结果，并通过 `if` 判断是否等于目标值 `x`。如果相等，就打印该表达式，并将计数变量 `count` 加 1。最终遍历所有可能的组合，得到全部符合要求的表达式及其数量。

二．代码

```python
#第五题代码
def Find_expression(x):
    count = 0

    for code in range(6561):
        n = code
        total = 0
        number = 1
        sign = 1
        expression = "1"

        for digit in range(2, 10):
            choice = n % 3
            n = n // 3

            if choice == 0:
                number = number * 10 + digit
                expression = expression + str(digit)

            else:
                total = total + sign * number
                number = digit

                if choice == 1:
                    sign = 1
                    expression = expression + "+" + str(digit)

                else:
                    sign = -1
                    expression = expression + "-" + str(digit)

        total = total + sign * number

        if total == x:
            print(expression, "=", x)
            count = count + 1

    return count
```

三．运行结果

```
Find_expression(100)

123-45-67+89 = 100
12-3-4+5-6+7+89 = 100
12+3+4+5-6-7+89 = 100
123+4-5+67-89 = 100
1+2+3-4+5+6+78+9 = 100
12+3-4+5+67+8+9 = 100
1+23-4+56+7+8+9 = 100
1+2+34-5+67-8+9 = 100
1+23-4+5+6+78-9 = 100
123+45-67+8-9 = 100
123-4-5-6-7+8-9 = 100

11
```

```
Find_expression(50)

1-2+34-5-67+89 = 50
1+2+3+4-56+7+89 = 50
1-2-34-5-6+7+89 = 50
1+2-34+5-6-7+89 = 50
12+3+4-56+78+9 = 50
1-2+3-45+6+78+9 = 50
1-23-4-5-6+78+9 = 50
1+2+34-5-6+7+8+9 = 50
1-2-3+4+56-7-8+9 = 50
1+2+34-56+78-9 = 50
1-23+4+5-6+78-9 = 50
1-2-3-4-5-6+78-9 = 50
1-2+34+5+6+7+8-9 = 50
1+2+3-4+56-7+8-9 = 50
12-3-4-5+67-8-9 = 50
1+2-3+4+56+7-8-9 = 50
12-3+45+6+7-8-9 = 50

17
```

```
Find_expression(10)

1+2+34-5+67-89 = 10
12+3+45-67+8+9 = 10
12+3-4-5-6-7+8+9 = 10
12-3+4-5-6+7-8+9 = 10
12-3-4+5+6-7-8+9 = 10
123-45-67+8-9 = 10
12-3-4+5-6+7+8-9 = 10
12+3+4+5-6-7+8-9 = 10
1-2-34-5+67-8-9 = 10
12+3+4-5+6+7-8-9 = 10
12-34+56-7-8-9 = 10
1-2+34-5+6-7-8-9 = 10

12
```

说明：这道题只有 def 和第一个 for 是自己写的，其他代码都是 gpt 写的，我实在是想不出来，数字中间如何插入+/-这步，还有用什么函数可以得到类似 12-34+56-78+9 这种结构的数据，想不到解法。感谢 gpt 的帮助。

5.2 代码全是 gpt 弄的。我只学了 gpt 给的教程